

Lab4-Assignment about Named Entity Recognition and Classification

This notebook describes the assignment of Lab 4 of the text mining course. We assume you have successfully completed Lab1, Lab2 and Lab3 as well. Especially Lab2 is important for completing this assignment.

Learning goals

- going from linguistic input format to representing it in a feature space
- working with pretrained word embeddings
- train a supervised classifier (SVM)
- evaluate a supervised classifier (SVM)
- learn how to interpret the system output and the evaluation results
- be able to propose future improvements based on the observed results

Credits

This notebook was originally created by [Marten Postma](#) and [Filip Ilievski](#) and adapted by Piek vossen

[Points: 18] Exercise 1 (NERC): Training and evaluating an SVM using CoNLL-2003

[4 point] a) Load the CoNLL-2003 training data using the *ConllCorpusReader* and create for both *train.txt* and *test.txt*:

```
[2 points] -a list of dictionaries representing the features for each
training instances, e.g.,
```
[
{'words': 'EU', 'pos': 'NNP'},
{'words': 'rejects', 'pos': 'VBZ'},
...
]
```

[2 points] -the NERC labels associated with each training instance,
e.g.,
dictionaries, e.g.,
```
[
'B-ORG',
'O',
```

```
....
]
\\
```

```
from nltk.corpus.reader import ConllCorpusReader
import numpy as np

Adapt the path to point to the CONLL2003 folder on your local
machine
train = ConllCorpusReader('CONLL2003', 'train.txt', ['words', 'pos',
'ignore', 'chunk'])
training_features = []
training_gold_labels = []

for token, pos, ne_label in train.iob_words():
 if token != '' and token != 'DOCSTART':
 a_dict = {
 "words": token,
 "pos": pos,
 "Is_upper": token.isupper(),
 "Is_lower": token.islower(),
 "Is_title": token.istitle(),
 "Is_number": token.isdigit(),
 }

 training_features.append(a_dict)
 training_gold_labels.append(ne_label)

print(training_features[0], training_features[1],
training_features[2])
print(training_gold_labels[0], training_gold_labels[1],
training_gold_labels[2])

{'words': 'EU', 'pos': 'NNP', 'Is_upper': True, 'Is_lower': False,
'Is_title': False, 'Is_number': False} {'words': 'rejects', 'pos':
'VBZ', 'Is_upper': False, 'Is_lower': True, 'Is_title': False,
'Is_number': False} {'words': 'German', 'pos': 'JJ', 'Is_upper':
False, 'Is_lower': False, 'Is_title': True, 'Is_number': False}
B-ORG 0 B-MISC

Adapt the path to point to the CONLL2003 folder on your local
machine
train = ConllCorpusReader('CONLL2003', 'test.txt', ['words', 'pos',
'ignore', 'chunk'])

test_features = []
test_gold_labels = []
```

```

for token, pos, ne_label in train.iob_words():
 if token != '' and token != 'DOCSTART':
 a_dict = {
 "words": token,
 "pos": pos,
 "Is_upper": token.isupper(),
 "Is_lower": token.islower(),
 "Is_title": token.istitle(),
 "Is_number": token.isdigit(),
 }

 test_features.append(a_dict)
 test_gold_labels.append(ne_label)

print(test_features[0], training_features[1], training_features[2])
print(test_gold_labels[0], test_gold_labels[1], test_gold_labels[2])

{'words': 'SOCCER', 'pos': 'NN', 'Is_upper': True, 'Is_lower': False,
 'Is_title': False, 'Is_number': False} {'words': 'rejects', 'pos':
 'VBZ', 'Is_upper': False, 'Is_lower': True, 'Is_title': False,
 'Is_number': False} {'words': 'German', 'pos': 'JJ', 'Is_upper':
 False, 'Is_lower': False, 'Is_title': True, 'Is_number': False}
0 0 B-LOC

print(training_features[0])

{'words': 'EU', 'pos': 'NNP', 'Is_upper': True, 'Is_lower': False,
 'Is_title': False, 'Is_number': False}

print(training_features[1:3])
print(training_gold_labels[0:10])

[{'words': 'rejects', 'pos': 'VBZ', 'Is_upper': False, 'Is_lower':
 True, 'Is_title': False, 'Is_number': False}, {'words': 'German',
 'pos': 'JJ', 'Is_upper': False, 'Is_lower': False, 'Is_title': True,
 'Is_number': False}]
['B-ORG', 'O', 'B-MISC', 'O', 'O', 'O', 'B-MISC', 'O', 'O', 'B-PER']

print(type(training_features), type(training_gold_labels))

<class 'list'> <class 'list'>

```

**[2 points] b) provide descriptive statistics about the training and test data:**

- How many instances are in train and test?
- Provide a frequency distribution of the NERC labels, i.e., how many times does each NERC label occur?

- Discuss to what extent the training and test data is balanced (equal amount of instances for each NERC label) and to what extent the training and test data differ?

Tip: you can use the following `Counter` functionality to generate frequency list of a list:

```
from collections import Counter

print(f'Length of training list: {len(training_features)}\n')
print(f'Length of test list: {len(test_features)}\n')
print(f'Frequency distribution of NERC labels in training list:
{Counter(training_gold_labels)}\n')
print(f'Frequency distribution of NERC labels in test list:
{Counter(test_gold_labels)}')
```

Length of training list: 203621

Length of test list: 46435

Frequency distribution of NERC labels in training list: Counter({'O': 169578, 'B-LOC': 7140, 'B-PER': 6600, 'B-ORG': 6321, 'I-PER': 4528, 'I-ORG': 3704, 'B-MISC': 3438, 'I-LOC': 1157, 'I-MISC': 1155})

Frequency distribution of NERC labels in test list: Counter({'O': 38323, 'B-LOC': 1668, 'B-ORG': 1661, 'B-PER': 1617, 'I-PER': 1156, 'I-ORG': 835, 'B-MISC': 702, 'I-LOC': 257, 'I-MISC': 216})

## Analysis of data statistics

- To get an overview of the dataset, we print the number of training and test instances with:
  - `print(f'Length of training list: {len(training_features)}\n')`
  - `print(f'Length of test list: {len(test_features)}\n')`

The output provides:

- Length of training list: 203621
- Length of test list: 46435

This output tells us that there are 203621 instances in the training dataset and 46435 instances in the test dataset, indicating that the training set is significantly larger than the test dataset.

- To analyze the frequency distribution of the NERC labels we use the `Counter` function to generate a frequency list of a list. We do this with:
  - `print(f'Frequency distribution of NERC labels in training list: {Counter(training_gold_labels)}\n')`
  - `print(f'Frequency distribution of NERC labels in test list: {Counter(test_gold_labels)}')`

The output provides:

- Frequency distribution of NERC labels in training list: Counter({'O': 169578, 'B-LOC': 7140, 'B-PER': 6600,

- 'B-ORG': 6321, 'I-PER': 4528, 'I-ORG': 3704, 'B-MISC': 3438, 'I-LOC': 1157, 'I-MISC': 1155}
- Frequency distribution of NERC labels in test list: Counter({'O': 38323, 'B-LOC': 1668, 'B-ORG': 1661, 'B-PER': 1617, 'I-PER': 1156, 'I-ORG': 835, 'B-MISC': 702, 'I-LOC': 257, 'I-MISC': 216})

The majority are labelled as "O" followed by labels such as "B-LOC", "B-ORG", "B-PER", "I-PER", "I-ORG", "B-MISC", "I-LOC", and "I-MISC".

- As we can see from the output that describes the frequency of each NERC label that the dataset is imbalanced. The "O" label occurs far more frequently than the other labels such as "B-LOC", "B-ORG", "B-PER", etc. However, the distribution in the test dataset follows a similar pattern to the training dataset. For example, the "O", "B-LOC", "B-ORG", "B-PER", "I-PER", "I-ORG", "B-MISC", "I-LOC", and "I-MISC" in the test dataset closely match the proportions of those in the training dataset. Indicating that the test dataset is representative of the training dataset.

**[2 points] c) Concatenate the train and test features (the list of dictionaries) into one list. Load it using the *DictVectorizer*. Afterwards, split it back to training and test.**

Tip: You've concatenated train and test into one list and then you've applied the DictVectorizer. The order of the rows is maintained. You can hence use an index (number of training instances) to split the `_array` back into train and test. Do NOT use: `from sklearn.model_selection import train_test_split` here.

```
from sklearn.feature_extraction import DictVectorizer
vec = DictVectorizer(sparse=False)

Concatenate training and test features
all_features = training_features + test_features

Apply vectorization
all_feature_matrix = vec.fit_transform(all_features)

Split the vectorized features back into training and testing sets
based on the number of training instances
num_train = len(training_features)
train_feature_matrix = all_feature_matrix[:num_train]
test_feature_matrix = all_feature_matrix[num_train:]

training_gold_labels = np.array(training_gold_labels)
test_gold_labels = np.array(test_gold_labels)

print(f"Train matrix: {train_feature_matrix}")
print(f"Test matrix: {test_feature_matrix}")
print()
print(f"Shape of training feature matrix:
{train_feature_matrix.shape}, {type(train_feature_matrix)}")
print(f"Shape of test feature matrix: {test_feature_matrix.shape},
{type(test_feature_matrix)}")
```

```
print(f'Shape of training labels: {training_gold_labels.shape}')
print(f'Shape of testing labels: {test_gold_labels.shape}')
```

```
Train matrix: [[0. 0. 0. ... 0. 0. 0.]
```

```
[1. 0. 0. ... 0. 0. 0.]
```

```
[0. 0. 1. ... 0. 0. 0.]
```

```
...
```

```
[0. 1. 0. ... 0. 0. 0.]
```

```
[0. 0. 1. ... 0. 0. 0.]
```

```
[0. 1. 0. ... 0. 0. 0.]]
```

```
Test matrix: [[0. 0. 0. ... 0. 0. 0.]
```

```
[0. 0. 0. ... 0. 0. 0.]
```

```
[0. 0. 0. ... 0. 0. 0.]
```

```
...
```

```
[0. 0. 0. ... 0. 0. 0.]
```

```
[0. 0. 0. ... 0. 0. 0.]
```

```
[0. 0. 0. ... 0. 0. 0.]]
```

```
Shape of training feature matrix: (203621, 27827), <class
'numpy.ndarray'>
```

```
Shape of test feature matrix: (20000, 27827), <class 'numpy.ndarray'>
```

```
Shape of training labels: (203621,)
```

```
Shape of testing labels: (46435,)
```

**[4 points] d) Train the SVM using the train features and labels and evaluate on the test data. Provide a classification report (sklearn.metrics.classification\_report).** The train (*lin\_clf.fit*) might take a while. On my computer, it took 1min 53s, which is acceptable. Training models normally takes much longer. If it takes more than 5 minutes, you can use a subset for training. Describe the results:

- Which NERC labels does the classifier perform well on? Why do you think this is the case?
- Which NERC labels does the classifier perform poorly on? Why do you think this is the case?

```
Check the shapes of the feature matrices and labels
```

```
print(f"Shape of training labels: {training_gold_labels.shape}")
```

```
print(f"Shape of testing labels: {test_gold_labels.shape}")
```

```
print(f"Shape of training feature matrix:
```

```
{train_feature_matrix.shape}")
```

```
print(f"Shape of testing feature matrix: {test_feature_matrix.shape}")
```

```
Shape of training labels: (203621,)
```

```
Shape of testing labels: (46435,)
```

```
Shape of training feature matrix: (203621, 27365)
```

```
Shape of testing feature matrix: (46435, 27365)
```

```
from sklearn import svm
```

```
from sklearn.metrics import classification_report
```

```

lin_clf = svm.LinearSVC(max_iter=50000)
lin_clf.fit(train_feature_matrix, training_gold_labels)
LinearSVC(max_iter=50000)
test_predictions = lin_clf.predict(test_feature_matrix)

print(f"Length of test_gold_labels: {len(test_gold_labels)}")
print(f"Length of test_predictions: {len(test_predictions)}")
print(f"Shape of test_feature_matrix: {test_feature_matrix.shape}")
print(f"Shape of test_gold_labels: {test_gold_labels.shape}")
print(f"Length of training_labels: {len(training_gold_labels)}")
print(f"Shape of training feature matrix:
{train_feature_matrix.shape}")
print(f"Original training size: {len(training_features)}")
print(f"Vectorized training matrix size:
{train_feature_matrix.shape[0]}")
train_feature_matrix.shape[0] == len(training_gold_labels)
All data matched up and is ready to be tested

```

```

Length of test_gold_labels: 46435
Length of test_predictions: 46435
Shape of test_feature_matrix: (46435, 27365)
Shape of test_gold_labels: (46435,)
Length of training_labels: 203621
Shape of training feature matrix: (203621, 27365)
Original training size: 203621
Vectorized training matrix size: 203621

```

True

```
print(classification_report(test_gold_labels, test_predictions))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| B-LOC        | 0.81      | 0.78   | 0.79     | 1668    |
| B-MISC       | 0.72      | 0.68   | 0.70     | 702     |
| B-ORG        | 0.80      | 0.53   | 0.63     | 1661    |
| B-PER        | 0.81      | 0.45   | 0.58     | 1617    |
| I-LOC        | 0.62      | 0.53   | 0.57     | 257     |
| I-MISC       | 0.57      | 0.59   | 0.58     | 216     |
| I-ORG        | 0.69      | 0.47   | 0.56     | 835     |
| I-PER        | 0.40      | 0.86   | 0.55     | 1156    |
| 0            | 0.98      | 0.99   | 0.98     | 38323   |
| accuracy     |           |        | 0.92     | 46435   |
| macro avg    | 0.71      | 0.65   | 0.66     | 46435   |
| weighted avg | 0.93      | 0.92   | 0.92     | 46435   |

## SVM classifier analysis

Looking at the overall performance, we observe a weighted average of 0.92 in f1 score. however, when the unweighted average performance is observed we come across an f1 score of 0.66 . there is a considerable gap of performance between weighted and macro average performance. This is an indication of a class domination within the classifier. When individual performances are observed, there is major inbalance between classes which confirms the initial thought. Class O performed very well with an f1 score of 0.98. B-LOC and B-MISC performed lower but still at a higher than average with f1 scores of 0.79 and 0.70 respectively. All other classes have lower than average performances. (B-ORG has 0.63 f1 score, B-PER has 0.58 f1 score, I-LOC has 0.57 f1 score, I-MISC has 0.58 f1 score, I-ORG has 0.56 f1 score and I-PER has 0.55 f1 score) This performane inbalance is caused by inbalance of data distribution. It is likely that overrepresentation of class O caused the classifier to overfit the entire model in favour of class O and against the other classes.

**[6 points] e) Train a model that uses the embeddings of these words as inputs. Test again on the same data as in 2d. Generate a classification report and compare the results with the classifier you built in 2d.**

```
import gensim

word_embedding_model =
gensim.models.KeyedVectors.load_word2vec_format(r"../Lab_2/Google_News
_files/GoogleNews-vectors-negative300.bin.gz", binary = True)

input_vectors = []
labels_gold = []

train = ConllCorpusReader(r'CONLL2003', 'train.txt', ['words', 'pos',
'ignore', 'chunk'])

for token, pos, ne_label in train.iob_words():
 if token != '' and token != 'DOCSTART':
 if token in word_embedding_model:
 vector = word_embedding_model[token]
 else:
 vector = [0]*300
 input_vectors.append(vector)
 labels_gold.append(ne_label)
print(input_vectors[0])
print(labels_gold[0])

[3.73535156e-02 -2.03125000e-01 2.12890625e-01 2.44140625e-01
 -2.85156250e-01 -3.44238281e-02 6.68945312e-02 -1.87500000e-01
 -3.90625000e-02 8.48388672e-03 -2.89062500e-01 -8.34960938e-02
 9.08203125e-02 -2.73437500e-01 -3.92578125e-01 -1.06445312e-01
 -6.59179688e-02 -9.94873047e-03 -5.41992188e-02 -4.17480469e-02
 2.63671875e-01 7.95898438e-02 1.50390625e-01 1.94335938e-01
 2.12890625e-01 9.86328125e-02 -3.35937500e-01 1.58203125e-01
 2.83203125e-01 2.33398438e-01 -1.19140625e-01 -2.30468750e-01]
```



|                 |                 |                 |                 |
|-----------------|-----------------|-----------------|-----------------|
| 2.61718750e-01  | 5.95703125e-02  | 2.61230469e-02  | -3.41796875e-01 |
| -1.54296875e-01 | 1.37695312e-01  | 9.86328125e-02  | 5.56640625e-02  |
| 3.14453125e-01  | 9.81445312e-02  | 1.58203125e-01  | 1.97265625e-01  |
| 2.27050781e-02  | -7.61718750e-02 | -2.96875000e-01 | 2.18750000e-01  |
| -3.59375000e-01 | 1.88476562e-01  | -1.08398438e-01 | 3.15856934e-03  |
| -5.83496094e-02 | 1.96289062e-01  | 1.28906250e-01  | -2.31445312e-01 |
| -3.92578125e-01 | 1.36108398e-02  | -2.94921875e-01 | -7.76367188e-02 |
| -1.85546875e-01 | -2.98828125e-01 | 1.40991211e-02  | 2.17285156e-02  |
| 1.29882812e-01  | -1.80664062e-01 | -1.56250000e-02 | 1.18164062e-01  |
| -2.67578125e-01 | -1.62109375e-01 | -1.20605469e-01 | 2.14843750e-01  |
| 1.88476562e-01  | 1.36718750e-01  | -2.98828125e-01 | -7.12890625e-02 |
| 2.12890625e-01  | 1.83593750e-01  | 2.28271484e-02  | 3.49609375e-01  |
| -3.82812500e-01 | -4.16015625e-01 | 3.14941406e-02  | 6.98242188e-02  |
| 7.91015625e-02  | 1.93359375e-01  | -5.05371094e-02 | -3.00781250e-01 |
| 1.40625000e-01  | 2.69531250e-01  | -4.80957031e-02 | -2.98828125e-01 |
| -2.59765625e-01 | 1.54296875e-01  | -7.66601562e-02 | -2.02148438e-01 |
| -5.49316406e-02 | -3.57421875e-01 | 4.21875000e-01  | -1.05957031e-01 |
| -5.78613281e-02 | -4.02832031e-02 | -1.35742188e-01 | 6.22558594e-02  |
| 7.51953125e-02  | 1.91406250e-01  | -1.43554688e-01 | -2.00195312e-01 |
| 1.55273438e-01  | -2.46093750e-01 | 2.09960938e-01  | -1.63085938e-01 |
| 1.42578125e-01  | 3.16406250e-01  | 2.35351562e-01  | 1.98242188e-01  |
| -1.35742188e-01 | 3.61328125e-02  | 2.98828125e-01  | 2.07031250e-01  |
| 7.76367188e-02  | -4.27246094e-02 | -2.46093750e-01 | -1.71875000e-01 |
| 4.51660156e-02  | -2.42187500e-01 | 3.97949219e-02  | -1.71661377e-03 |
| -5.39062500e-01 | -2.73437500e-02 | 1.44531250e-01  | 2.00195312e-01  |
| -1.85546875e-01 | 5.95703125e-02  | -2.11914062e-01 | -2.26562500e-01 |
| -5.00488281e-02 | -2.23632812e-01 | 2.85156250e-01  | -3.06640625e-01 |
| 2.26562500e-01  | -4.85839844e-02 | -2.05078125e-01 | 1.02050781e-01  |
| -1.61132812e-02 | -1.33789062e-01 | 2.67578125e-01  | 1.06933594e-01  |
| -2.91015625e-01 | -1.19140625e-01 | 1.52343750e-01  | 1.79443359e-02  |
| -4.95605469e-02 | -2.08007812e-01 | 3.55468750e-01  | -1.82617188e-01 |
| -9.66796875e-02 | -4.02832031e-02 | 1.16699219e-01  | 3.83300781e-02  |
| -2.42187500e-01 | -1.11816406e-01 | -9.58251953e-03 | -4.12109375e-01 |
| 1.66015625e-01  | -1.63085938e-01 | 1.02539062e-01  | 6.93359375e-02  |
| -7.95898438e-02 | 8.10546875e-02  | -1.87500000e-01 | -1.38671875e-01 |
| 8.78906250e-02  | 8.59375000e-02  | -1.20605469e-01 | -4.02343750e-01 |
| 3.90625000e-02  | -2.08984375e-01 | 1.02050781e-01  | -1.07421875e-01 |
| 4.61425781e-02  | 1.69677734e-02  | 3.47656250e-01  | -6.68945312e-02 |
| 2.09960938e-01  | 1.98242188e-01  | 8.54492188e-02  | -8.15429688e-02 |
| -1.47460938e-01 | -9.13085938e-02 | -1.54296875e-01 | -2.61718750e-01 |
| -2.28271484e-02 | -8.30078125e-02 | 3.26171875e-01  | 3.73046875e-01  |
| 3.41796875e-02  | -3.90625000e-01 | -6.39648438e-02 | -3.41796875e-01 |
| -1.19628906e-01 | -1.08886719e-01 | -5.73730469e-02 | 2.28515625e-01  |
| -3.20434570e-04 | 1.31835938e-01  | 2.73437500e-01  | -2.92968750e-01 |
| -5.81054688e-02 | -2.18750000e-01 | -5.12695312e-02 | -1.14257812e-01 |
| -9.66796875e-02 | 5.00000000e-01  | -1.44531250e-01 | -3.61328125e-02 |
| 2.33459473e-03  | -4.61425781e-02 | 2.50000000e-01  | -5.81054688e-02 |
| -3.35937500e-01 | 2.01416016e-02  | -9.61914062e-02 | 3.08593750e-01  |
| -1.52343750e-01 | -1.77734375e-01 | 3.96484375e-01  | 2.23632812e-01  |

```

4.43359375e-01 -1.26953125e-01 -9.52148438e-02 4.60815430e-03
2.10937500e-01 -1.49414062e-01 2.85644531e-02 1.55273438e-01
5.02929688e-02 1.70898438e-01 -1.84326172e-02 -2.71484375e-01
-1.77001953e-02 1.29882812e-01 1.20605469e-01 5.29785156e-02
2.53906250e-01 3.51562500e-02 -3.12500000e-01 -2.17773438e-01
1.07421875e-02 -1.92382812e-01 2.94189453e-02 4.98046875e-02
-6.64062500e-02 3.80859375e-01 4.33349609e-03 -7.12890625e-02
4.16015625e-01 -1.49414062e-01 3.16406250e-01 -1.05957031e-01
-3.84521484e-03 1.42578125e-01 -5.49316406e-02 1.84570312e-01
2.37304688e-01 -1.44531250e-01 -3.04687500e-01 -3.78906250e-01
-2.85156250e-01 -1.75781250e-01 -2.40478516e-02 -5.59082031e-02
1.66992188e-01 5.27343750e-02 -2.65625000e-01 9.86328125e-02
1.12792969e-01 1.72119141e-02 -1.25000000e-01 1.46484375e-01
-1.19628906e-01 1.95312500e-01 -2.88085938e-02 2.61718750e-01
4.61425781e-02 1.03515625e-01 -1.31835938e-01 3.10546875e-01
9.96093750e-02 2.55859375e-01 -2.99072266e-03 1.63574219e-02
-2.45361328e-02 6.78710938e-02 -1.41601562e-01 4.41406250e-01
8.44726562e-02 1.57226562e-01 2.59765625e-01 -5.88378906e-02]

```

B-ORG

```

print(len(input_vectors))
print(len(labels_gold))
print(type(input_vectors), type(labels_gold))
print()
print(f'Distribution of labels in data: {Counter(labels_gold)}')
input_vectors = np.array(input_vectors)
labels_gold = np.array(labels_gold)
print(input_vectors.shape, labels_gold.shape)

```

203621

203621

<class 'numpy.ndarray'> <class 'numpy.ndarray'>

```

Distribution of labels in data: Counter({'0': 169578, 'B-L0C': 7140,
'B-PER': 6600, 'B-ORG': 6321, 'I-PER': 4528, 'I-ORG': 3704, 'B-MISC':
3438, 'I-L0C': 1157, 'I-MISC': 1155})
(203621, 300) (203621,)

```

```
lin_clf.fit(input_vectors, labels_gold)
```

```
LinearSVC(max_iter=50000)
```

```
test_input_vectors = []
```

```
test_labels_gold = []
```

```

for token, pos, ne_label in train.iob_words():
 if token != '' and token != 'DOCSTART':
 if token in word_embedding_model:
 vector = word_embedding_model[token]
 else:
 vector = [0] * 300

```

```

 test_input_vectors.append(vector)
 test_labels_gold.append(ne_label)

test_input_vectors = np.array(test_input_vectors)
test_labels_gold = np.array(test_labels_gold)
test_predictions = lin_clf.predict(test_input_vectors)

print(classification_report(test_labels_gold, test_predictions))

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| B-LOC        | 0.81      | 0.81   | 0.81     | 7140    |
| B-MISC       | 0.78      | 0.70   | 0.74     | 3438    |
| B-ORG        | 0.70      | 0.63   | 0.66     | 6321    |
| B-PER        | 0.77      | 0.72   | 0.74     | 6600    |
| I-LOC        | 0.69      | 0.56   | 0.62     | 1157    |
| I-MISC       | 0.69      | 0.40   | 0.51     | 1155    |
| I-ORG        | 0.66      | 0.41   | 0.50     | 3704    |
| I-PER        | 0.61      | 0.60   | 0.61     | 4528    |
| 0            | 0.97      | 1.00   | 0.98     | 169578  |
| accuracy     |           |        | 0.94     | 203621  |
| macro avg    | 0.74      | 0.65   | 0.69     | 203621  |
| weighted avg | 0.93      | 0.94   | 0.93     | 203621  |

## Evaluating the results of embeddings model

Embeddings model improves weighted avg f1 score by 0.01 (0.93 vs 0.92) by improving the recall by 0.02 (0.94 vs 0.92). Both classifiers performed same weighted avg precision (0.93). Moving to macro avg, the f1 score is improved (0.69 vs 0.66) by improving the precision (0.74 vs 0.71). Both models perform the same recall in this class (0.65). Moving to class O, the f1 score remains the same (0.98). This is a result of decreasing precision (0.97 vs 0.98) and increasing recall performance (1.00 vs 0.99). The model again is likely to overfit the data in favour of Class O. Moving to I-PER, the f1 score improved (0.61 vs 0.55) by improving the precision score (0.61 vs 0.40). The recall performance in this class decreased significantly (0.60 vs 0.86). Moving to I-ORG class, the f1 score decreased (0.50 vs 0.56) due to a decrease in both precision (0.66 vs 0.69) and recall performances (0.41 vs 0.47). Moving to I-MISC class, the f1 score decreased again (0.51 vs 0.58). This is a result of increasing precision (0.69 vs 0.57) and decreasing recall performance (0.40 vs 0.59). Moving to I-LOC class, the f1 score increases (0.62 vs 0.57). This is a result of increasing both the precision performance (0.69 vs 0.62) and recall performance (0.56 vs 0.53). Moving to B-PER class, the f1 score increases (0.74 vs 0.58) by increasing the recall rate (0.72 vs 0.45), the precision slightly decreases (0.77 vs 0.81). Moving to B-ORG class, we observe that the f1 score increases (0.66 v 0.63) by increasing the recall rate (0.63 vs 0.53), the precision slightly decreases again (0.70 vs 0.80). Moving to B-MISC class the f1 score increased (0.74 vs 0.70). This is a result of increasing both recall (0.70 vs 0.68) and precision scores (0.78 vs 0.72). Moving to the final class of B-LOC, the f1 score increases (0.81 vs 0.79). This is a result of increasing recall score (0.81 vs 0.78), the precision score remains the same (0.81).

Overall, The embeddings model performs better at beginning words. In inside words, the embeddings model perform better at location and person categories and performs worse in organization and miscellaneous classes. In "other words" category, both models perform the same. The embeddings model also creates a more balanced precision and recall performance.

## [Points: 10] Exercise 2 (NERC): feature inspection using the Annotated Corpus for Named Entity Recognition

[6 points] a. Perform the same steps as in the previous exercise. Make sure you end up for both the training part (*df\_train*) and the test part (*df\_test*) with:

- the features representation using **DictVectorizer**
- the NERC labels in a list

Please note that this is the same setup as in the previous exercise:

- load both train and test using:
  - list of dictionaries for features
  - list of NERC labels
- combine train and test features in a list and represent them using one hot encoding
- train using the training features and NERC labels

```
import pandas as pd

Adapt the path to point to your local copy of NERC_datasets
path = 'nerc_datasets/ner_v2.csv'
kaggle_dataset = pd.read_csv(path, on_bad_lines='skip')

len(kaggle_dataset)

1050795

df_train = kaggle_dataset[:100000]
df_test = kaggle_dataset[100000:120000]
print(len(df_train), len(df_test))

100000 20000

train_features = df_train[['word', 'pos',
'shape']].to_dict(orient='records')
test_features = df_test[['word', 'pos',
'shape']].to_dict(orient='records')

train_labels = df_train['tag'].tolist()
test_labels = df_test['tag'].tolist()

print(train_features[0])
print(train_labels[0])
```

```

vec = DictVectorizer(sparse=False)
all_features = train_features + test_features
all_feature_matrix = vec.fit_transform(all_features)

train_feature_matrix = all_feature_matrix[:len(train_features)]
test_feature_matrix = all_feature_matrix[len(train_features):]

print(f"Shape of training feature matrix:
{train_feature_matrix.shape}")
print(f"Shape of testing feature matrix: {test_feature_matrix.shape}")

{'word': 'Thousands', 'pos': 'NNS', 'shape': 'capitalized'}
0
Shape of training feature matrix: (100000, 11908)
Shape of testing feature matrix: (20000, 11908)

```

**[4 points] b. Train and evaluate the model and provide the classification report:**

- use the SVM to predict NERC labels on the test data
- evaluate the performance of the SVM on the test data

Analyze the performance per NERC label.

```

lin_clf.fit(train_feature_matrix, train_labels)
LinearSVC(max_iter=50000)
test_predictions = lin_clf.predict(test_feature_matrix)
print(classification_report(test_labels, test_predictions))

```

|       | precision | recall | f1-score | support |
|-------|-----------|--------|----------|---------|
| B-art | 1.00      | 0.50   | 0.67     | 4       |
| B-eve | 0.00      | 0.00   | 0.00     | 0       |
| B-geo | 0.81      | 0.76   | 0.78     | 741     |
| B-gpe | 0.97      | 0.92   | 0.95     | 296     |
| B-nat | 1.00      | 0.50   | 0.67     | 8       |
| B-org | 0.67      | 0.57   | 0.61     | 397     |
| B-per | 0.81      | 0.54   | 0.65     | 333     |
| B-tim | 0.91      | 0.76   | 0.83     | 393     |
| I-art | 0.00      | 0.00   | 0.00     | 0       |
| I-eve | 0.00      | 0.00   | 0.00     | 0       |
| I-geo | 0.74      | 0.50   | 0.60     | 156     |
| I-gpe | 1.00      | 0.50   | 0.67     | 2       |
| I-nat | 0.80      | 1.00   | 0.89     | 4       |
| I-org | 0.65      | 0.45   | 0.53     | 321     |
| I-per | 0.43      | 0.88   | 0.58     | 319     |
| I-tim | 0.41      | 0.08   | 0.14     | 108     |
| 0     | 0.98      | 0.99   | 0.99     | 16918   |

|              |      |      |      |       |
|--------------|------|------|------|-------|
| accuracy     |      |      | 0.94 | 20000 |
| macro avg    | 0.66 | 0.53 | 0.56 | 20000 |
| weighted avg | 0.95 | 0.94 | 0.94 | 20000 |

```

/Users/robertostoica/Desktop/anaconda3/envs/Text_Mining_python/lib/
python3.12/site-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in
labels with no true samples. Use `zero_division` parameter to control
this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
/Users/robertostoica/Desktop/anaconda3/envs/Text_Mining_python/lib/
python3.12/site-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in
labels with no true samples. Use `zero_division` parameter to control
this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
/Users/robertostoica/Desktop/anaconda3/envs/Text_Mining_python/lib/
python3.12/site-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in
labels with no true samples. Use `zero_division` parameter to control
this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))

```

## Evaluation of SVM model

To assess the model's accuracy and performance, we used `classification_report` which provides the precision, recall, F1-score, and support score for the NERC labels. The overall performance of the SVM classifier:

- Accuracy F1-score: 94%, indicates that the model performs overall well
- Macro Average F1-score: 56%, indicates that performance varies across different labels. Suggests poor performance on less frequent labels—which drags the score down.
- Weighted Average F1-score: 94%, indicates that the model performs well on frequent labels.

Noticeably, the "O" label performs extremely well with a 98% precision, 99% recall, and 0.99 F1-score. Other high-performing labels are "B-geo", "B-gpe", and "B-tim".

Low-performing labels such as "B-art" and "I-tim" indicate that the model struggled to detect them and have a low support score. Additionally, some labels have zero support which indicates that they do not appear at all in the test dataset, for example "B-eve" and "I-art". Because these labels do not exist in the test dataset, the model is unable to predict them, leading to undefined precision, recall, F1-score values.

To summarise, there seems to be a clear correlation between the support score and the overall performance of the labels. Labels with a higher support score tend to have a higher precision,

recall, and F1-score compared to the labels with a low support score. This is likely due to the lack of sufficient training data for the less frequent labels, preventing the model from learning strong patterns to classify them accurately.

## End of this notebook